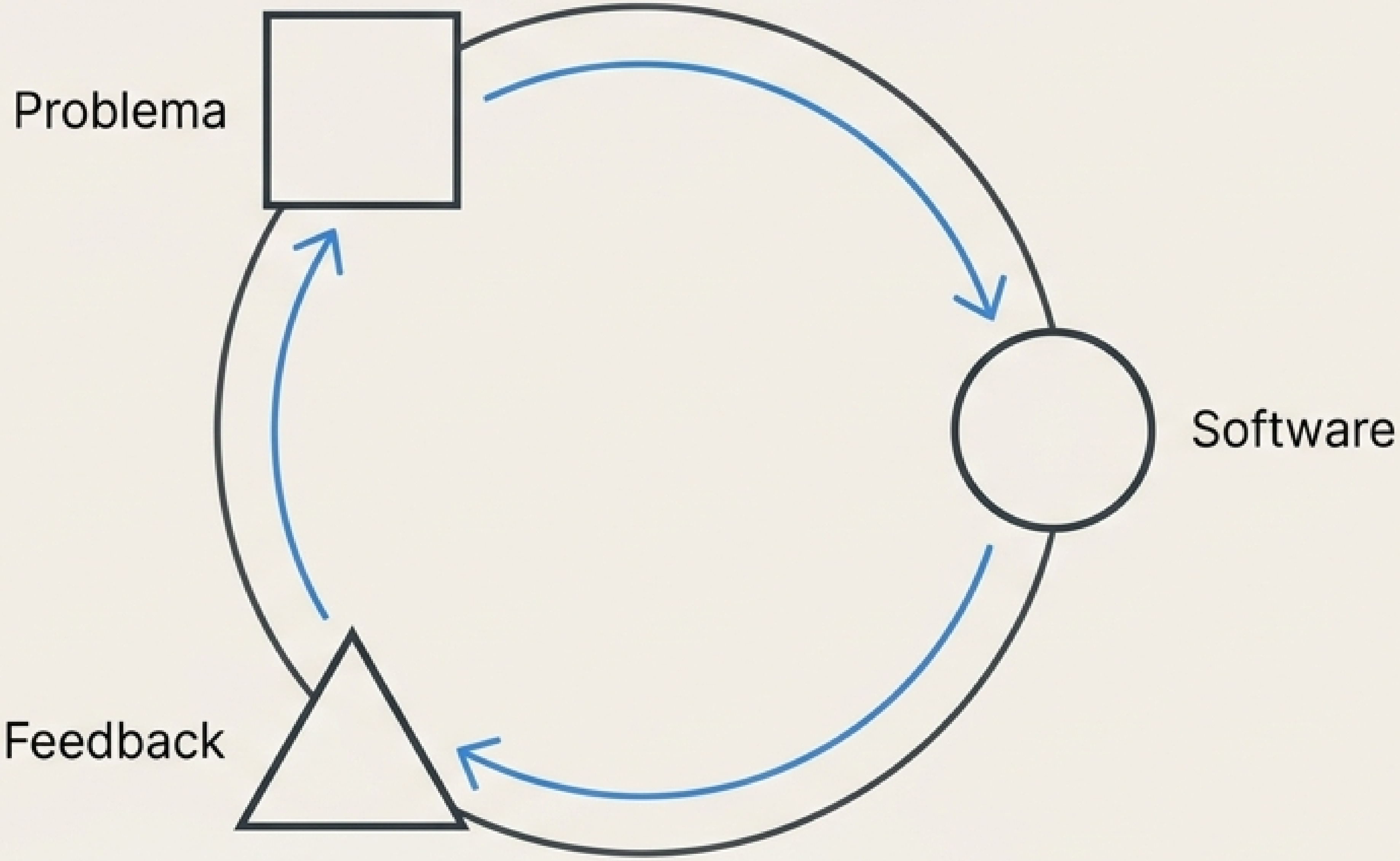
O Sistema Contínuo: Da Engenharia ao Aprendizado



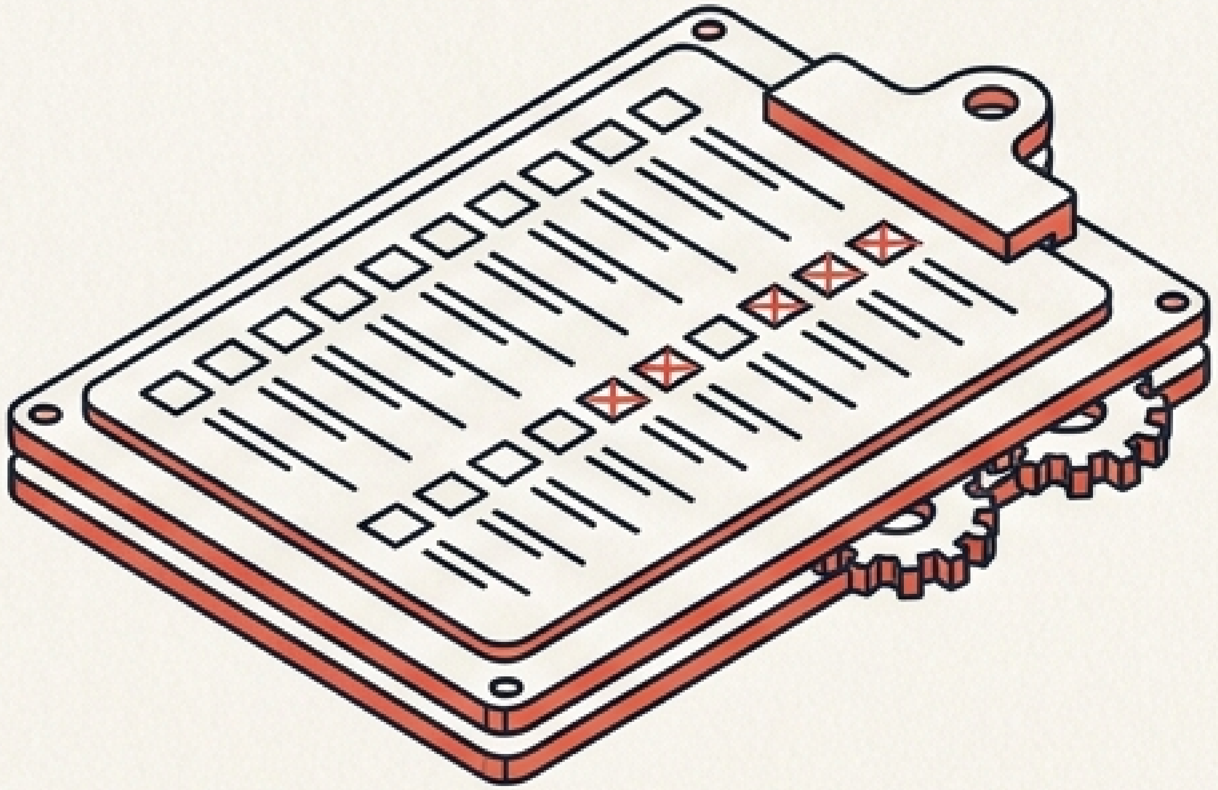
Uma equipe de software de alto desempenho não é uma fábrica de código. É um sistema que transforma problemas em pequenas mudanças, entrega essas mudanças e utiliza o feedback para decidir o próximo passo.

O objetivo não é manter todos ocupados cumprindo listas de tarefas. O objetivo é fazer o trabalho fluir até o usuário e aprender com o resultado.

A ORIGEM DO TRABALHO: PROBLEMAS > TAREFAS

A FÁBRICA DE TAREFAS

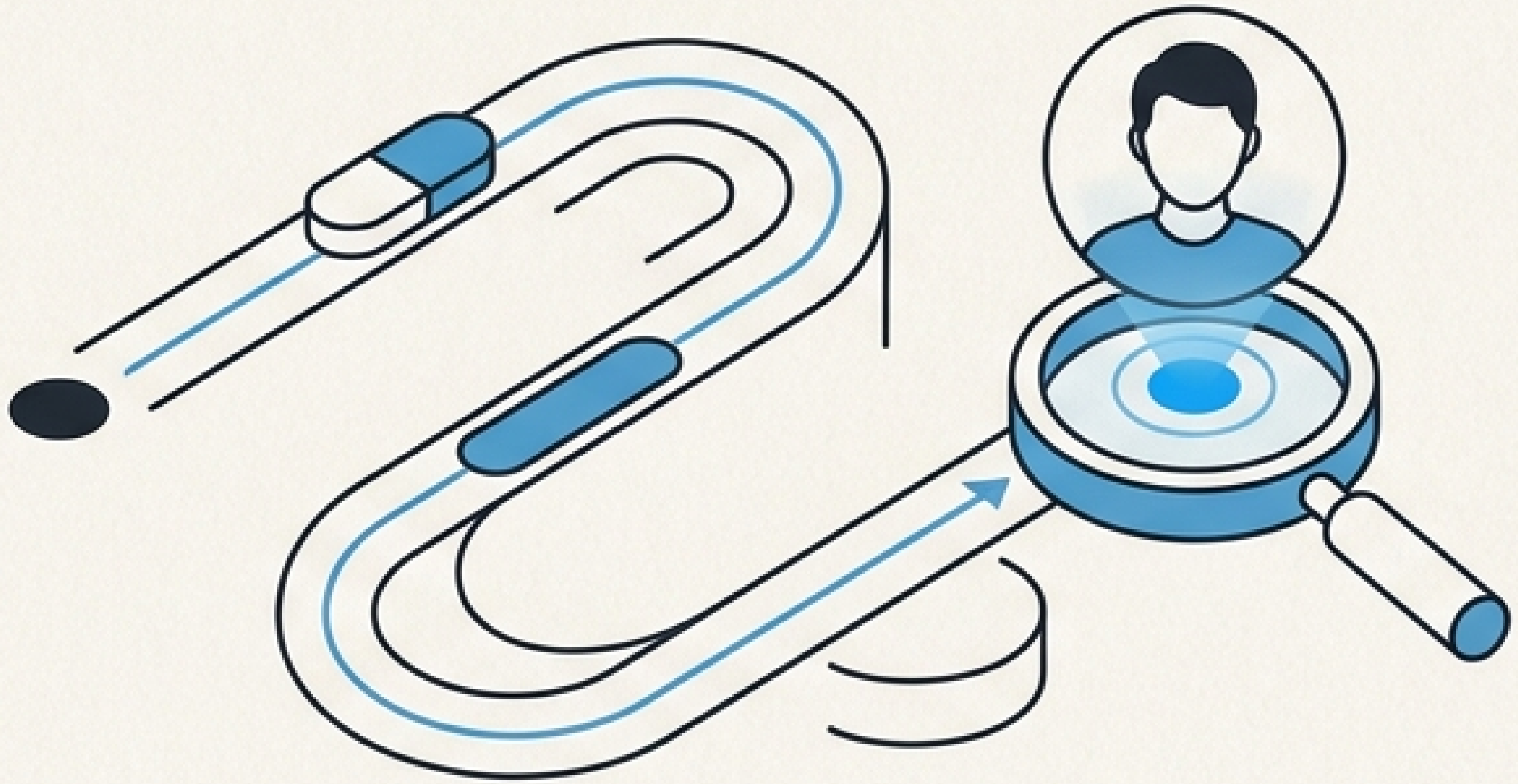
"Como vamos construir tudo isso?"



A instrução "Implementar sistema" é abstrata.
Não define o comportamento ou as hipóteses.

O SISTEMA DE SOLUÇÕES

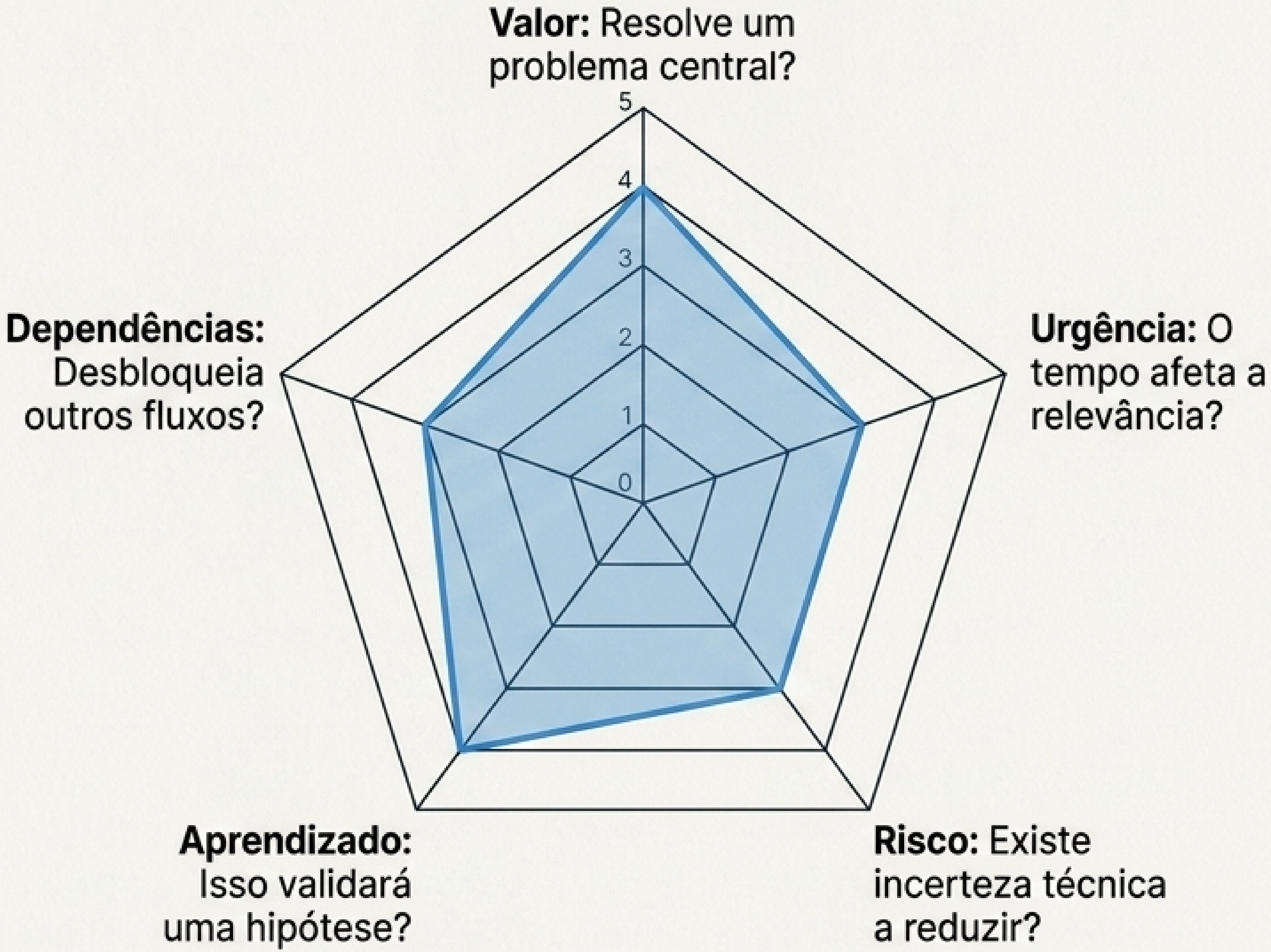
"Qual problema do usuário estamos resolvendo?"



A TRANSFORMAÇÃO NECESSÁRIA

PROBLEMA NO MUNDO REAL → **COMPORTAMENTO DESEJADO NO SISTEMA** → **PEQUENA MUDANÇA NO CÓDIGO**

O BACKLOG É DINÂMICO, NÃO UM CONTRATO



A capacidade de engenharia é o recurso mais limitado. Priorizar significa decidir exatamente onde focar essa capacidade.

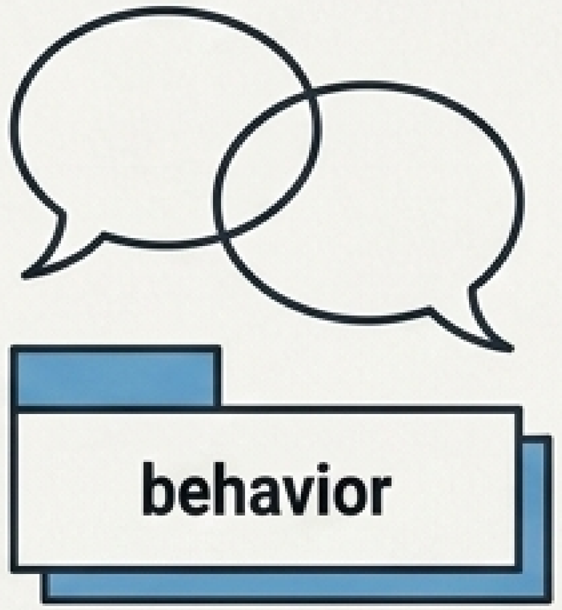
❗

Dizer "não agora" é uma das decisões de engenharia mais importantes que a equipe pode tomar.

Isso permite que a equipe se concentre nas iniciativas de maior impacto e evite o desperdício de recursos em tarefas menos críticas.

HISTÓRIAS DE USUÁRIO: COMPORTAMENTO, NÃO ESPECIFICAÇÃO

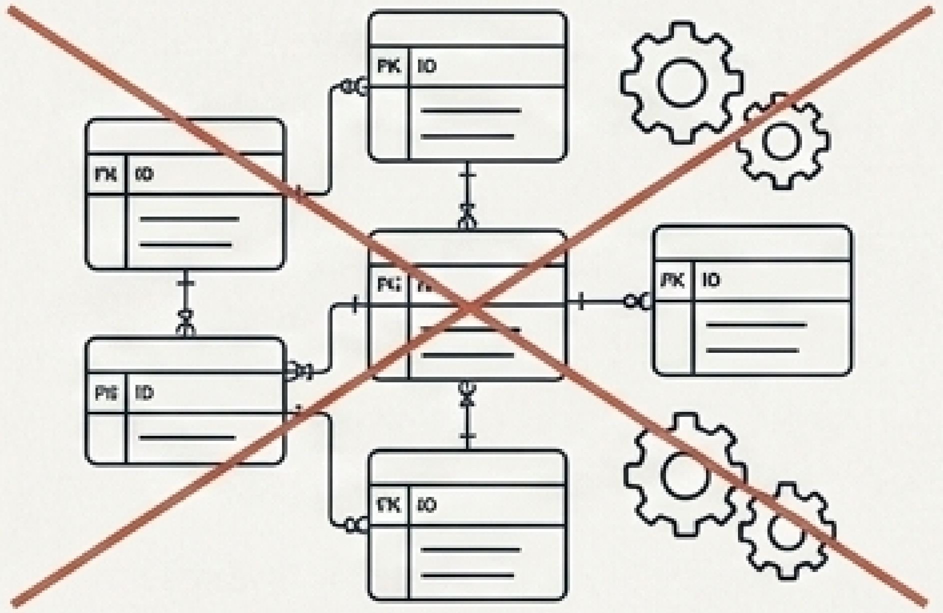
O QUE É (O FOCO)



- Uma unidade de comportamento desejado.
- O início de uma conversa técnica.

Exemplo: “Como cliente, quero pagar minha reserva com cartão.”

O QUE NÃO É (A ARMADILHA)

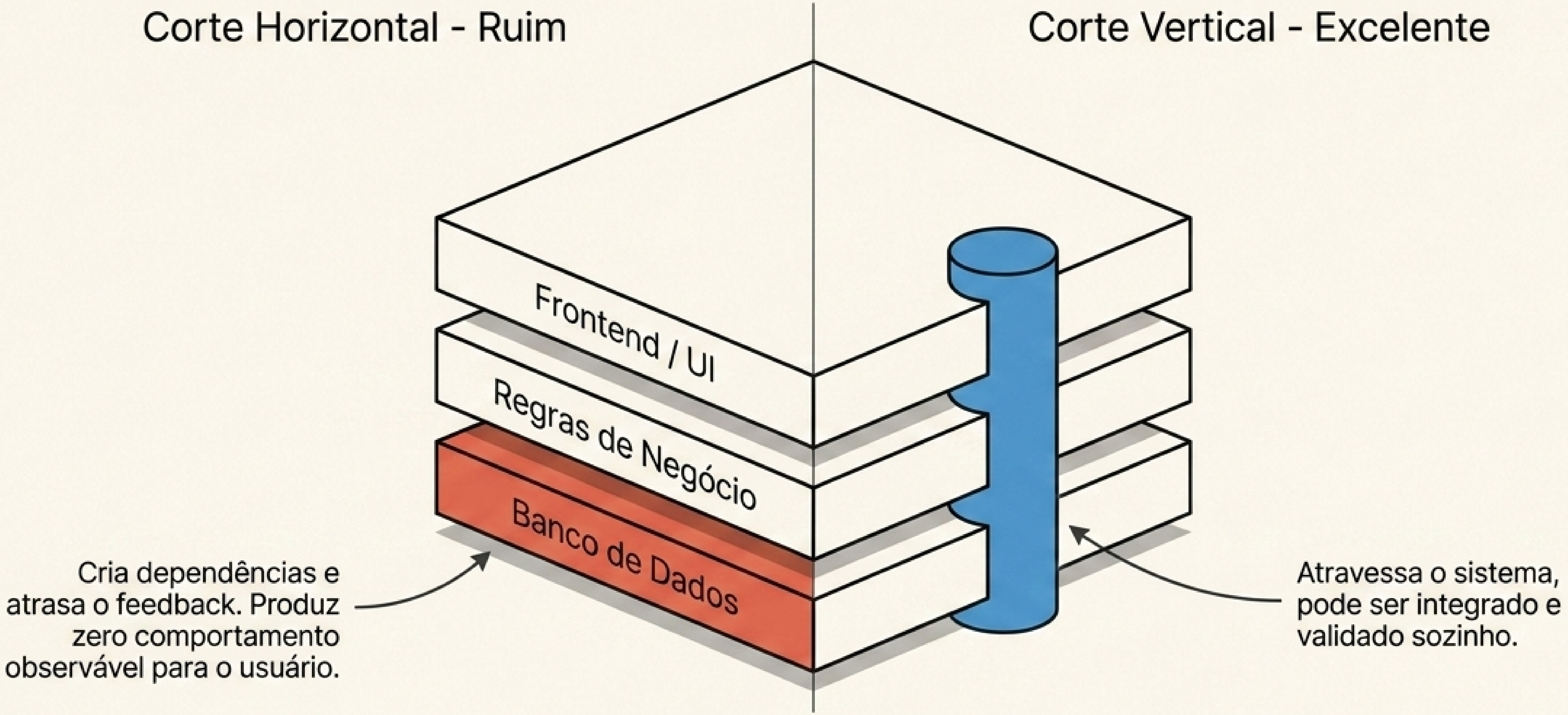


- Uma especificação rígida.
- Decisões prematuras de arquitetura.

Exemplo: “Criar tabela tb_payments no PostgreSQL com arquitetura de microsserviços.”

A história diz o que o sistema deve fazer. Como isso será implementado (**banco de dados, arquitetura, classes**) é o trabalho da engenharia, decidido durante a conversa.

A Anatomia de uma Mudança: Fatiamento Vertical

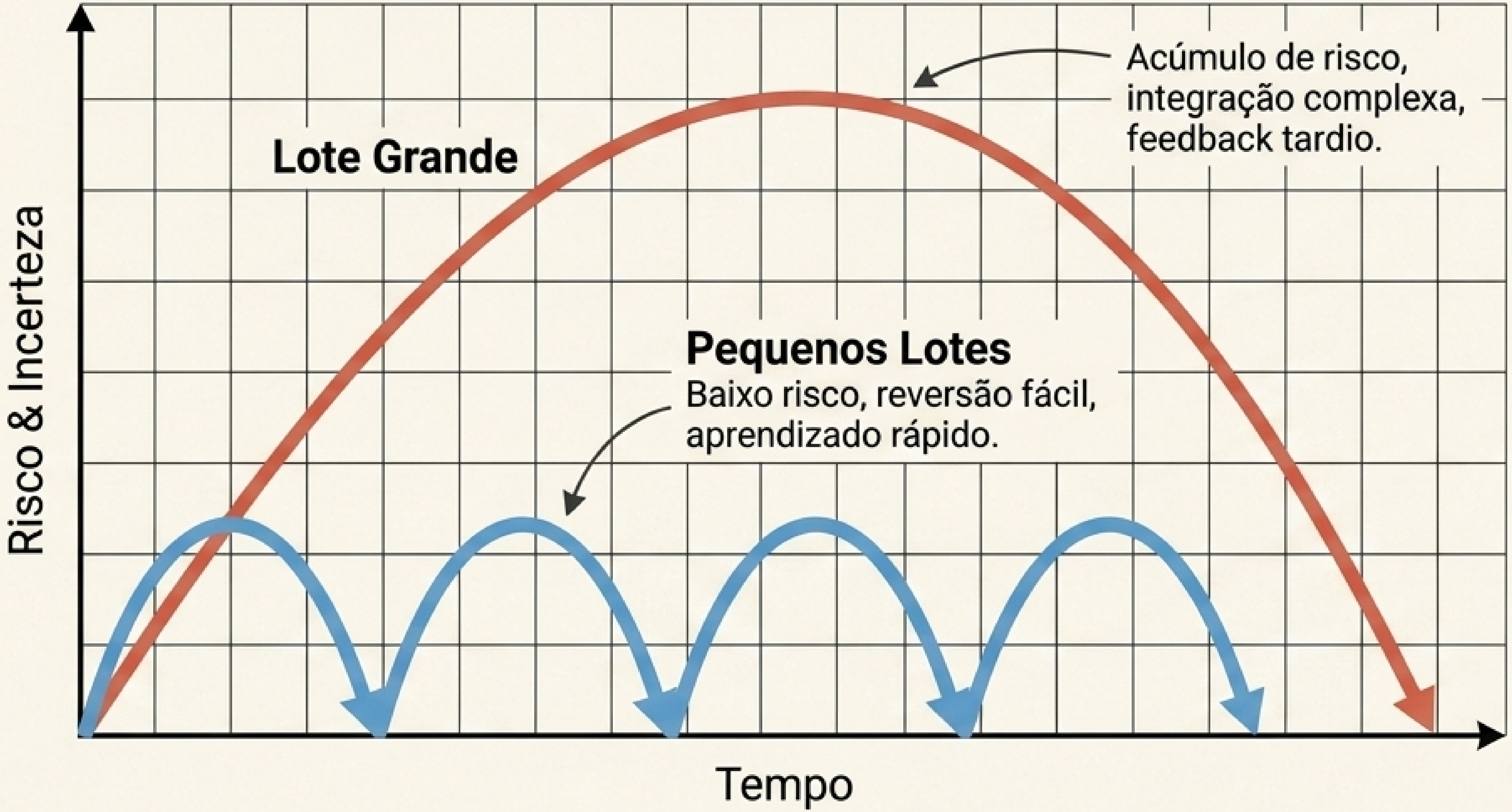

Corte Horizontal - Ruim
Corte Vertical - Excelente
Frontend / UI
Regras de Negócio
Banco de Dados

Cria dependências e atrasa o feedback. Produz zero comportamento observável para o usuário.

Atravessa o sistema, pode ser integrado e validado sozinho.

A Regra do Fatiamento: Uma pequena mudança deve produzir comportamento observável.

A Física dos Pequenos Lotes



Manifesto Ágil

"Deliver working software frequently..."

Engenharia do Google

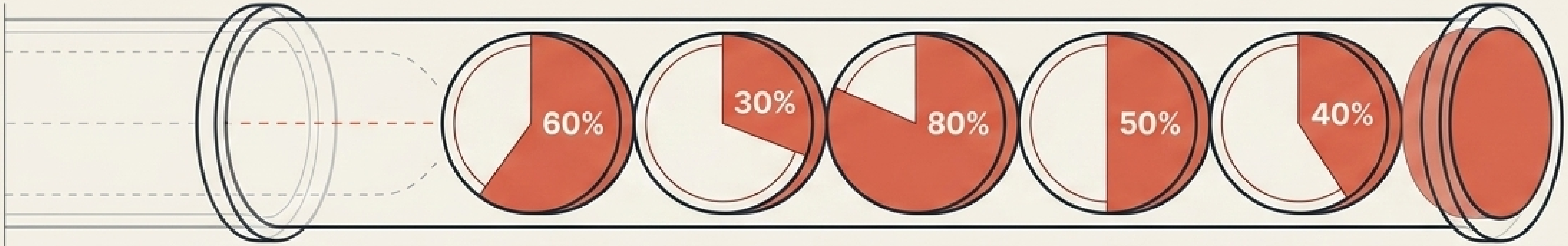
"Erre para o lado de escrever mudanças menores do que grandes."

Se uma mudança pode ser feita de maneira menor sem perder coerência, sempre vale a pena torná-la menor.

WIP: A Ilusão do Progresso Contínuo

Começar não cria valor. Terminar, sim.

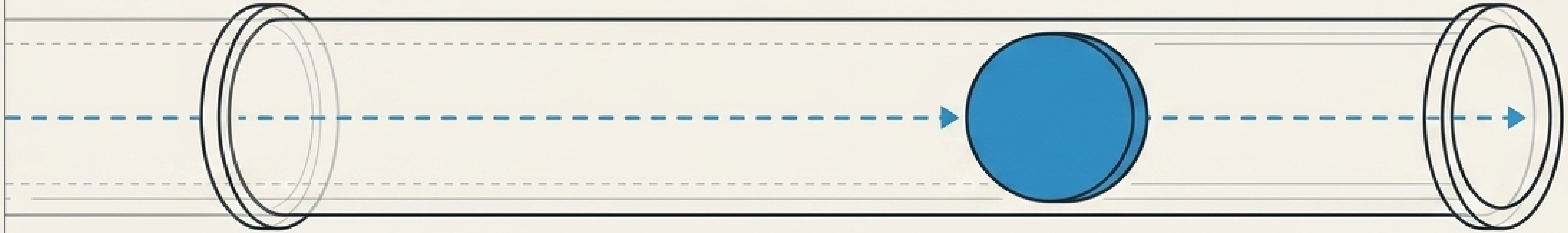
Equipe A (WIP Alto)



Valor entregue ao usuário = 0

Excesso de trabalho simultâneo cria filas, interrupções e mudança de contexto.

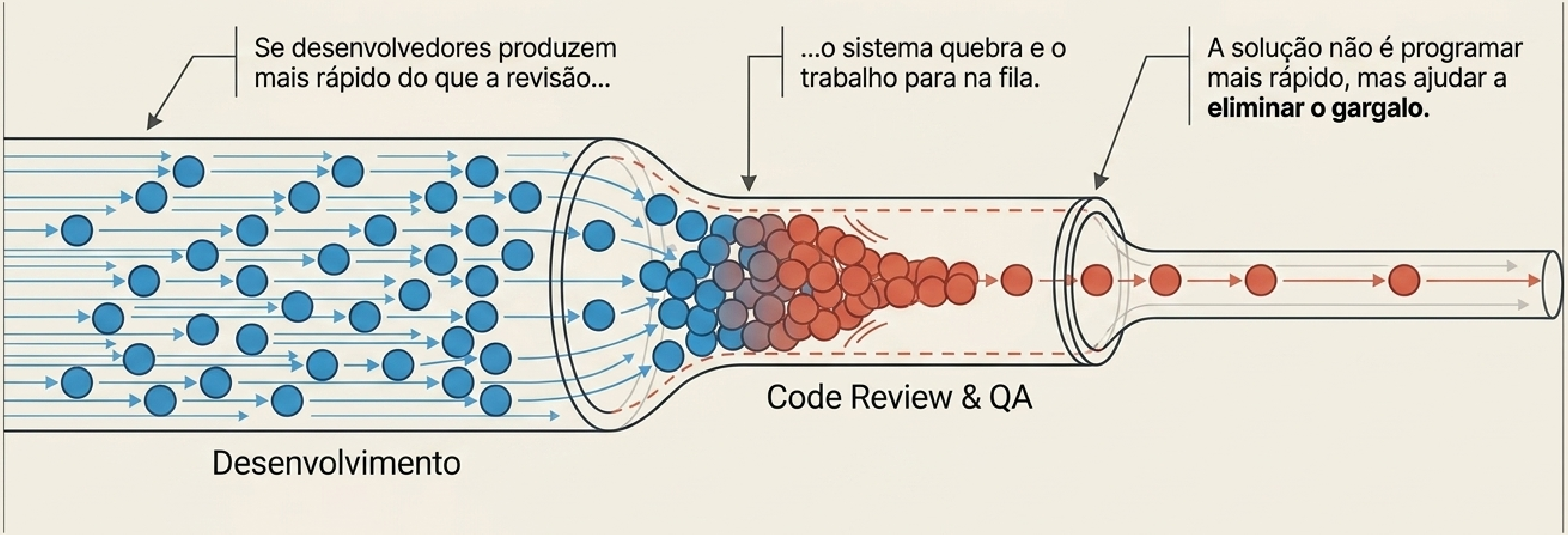
Equipe B (WIP Controlado)



Valor entregue ao usuário = 100%

Foco no fluxo. Fecha-se um ciclo antes de abrir outro.

Fluxo do Sistema vs. Ocupação Individual



Se desenvolvedores produzem mais rápido do que a revisão...

...o sistema quebra e o trabalho para na fila.

A solução não é programar mais rápido, mas ajudar a **eliminar o gargalo.**

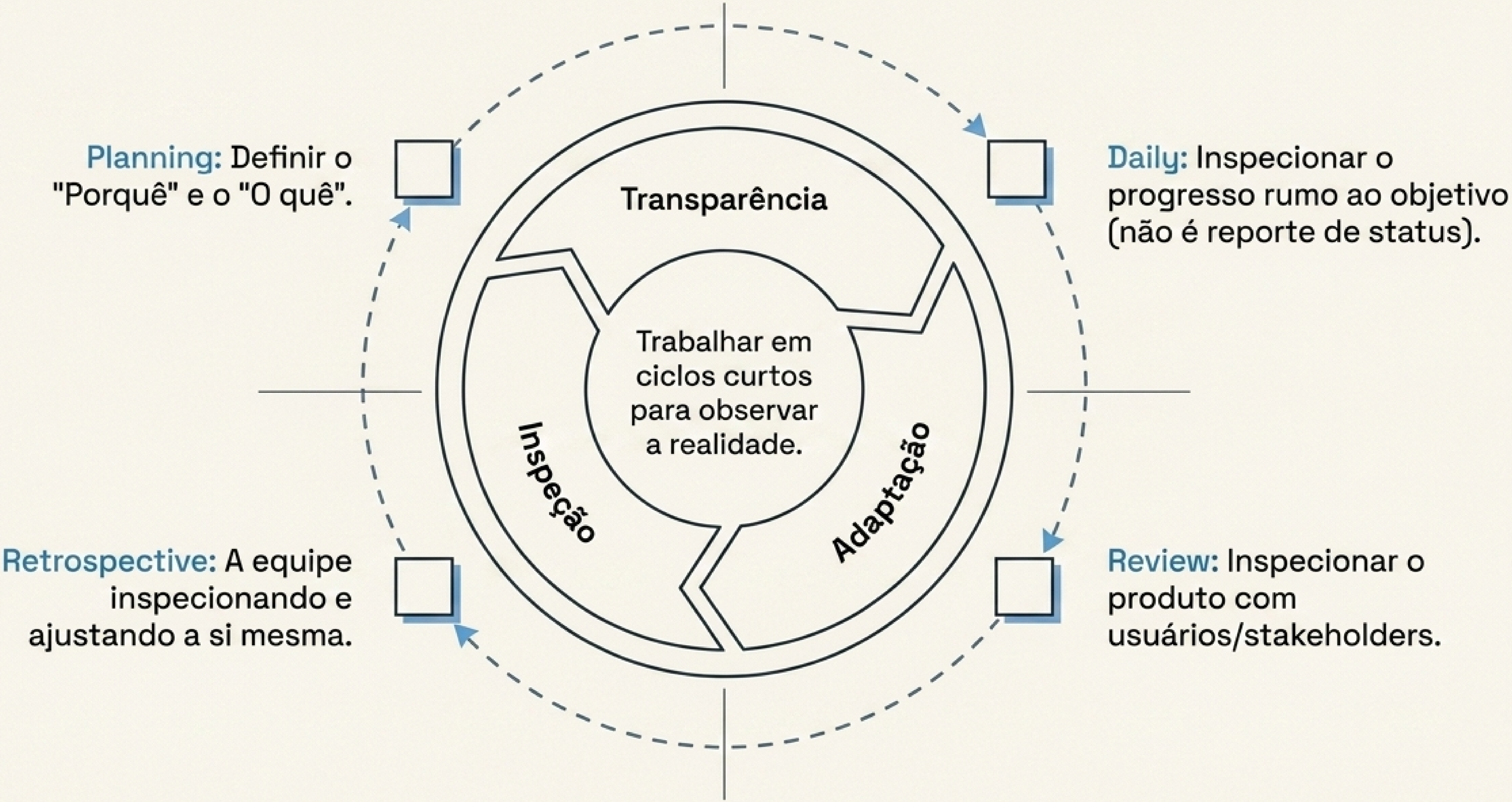
Ocupação de 100% não significa eficiência.
O objetivo é otimizar o fluxo de ponta a ponta, não a produtividade isolada.

Descodificando o Ecossistema Ágil

Conceito	O que é	Foco Principal
Agile	Valores e Princípios	Lidar com incerteza e mudança.
Scrum	Framework Organizacional	Cadência, papéis e organização empírica.
Kanban	Sistema Visual de Fluxo	Limitar WIP e otimizar gargalos.
XP (Extreme Programming)	Práticas de Engenharia	Testes, Refatoração, Código Limpo, CI/CD.

Atenção: Agile NÃO significa usar Jiira, ter Story Points, ou fazer Daily standups como reporte de status.

Scrum: O Motor Empírico



Transparência
Inspeção
Adaptação

Trabalhar em ciclos curtos para observar a realidade.

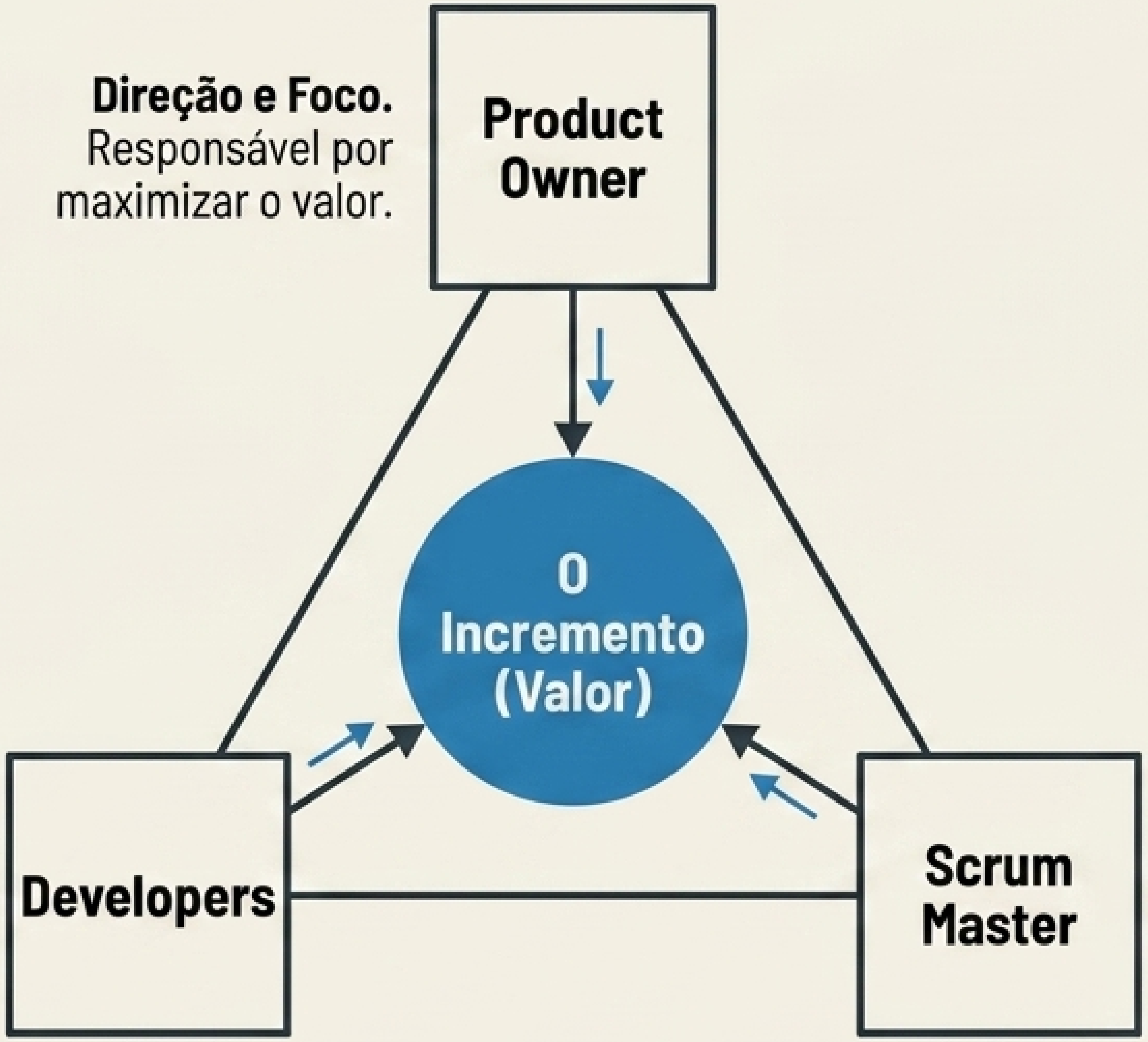
Planning: Definir o "Porquê" e o "O quê".

Daily: Inspecionar o progresso rumo ao objetivo (não é reporte de status).

Review: Inspecionar o produto com usuários/stakeholders.

Retrospective: A equipe inspecionando e ajustando a si mesma.

A Dinâmica da Equipe Scrum


Product Owner
O Incremento (Valor)
Developers
Scrum Master

Direção e Foco. Responsável por maximizar o valor.

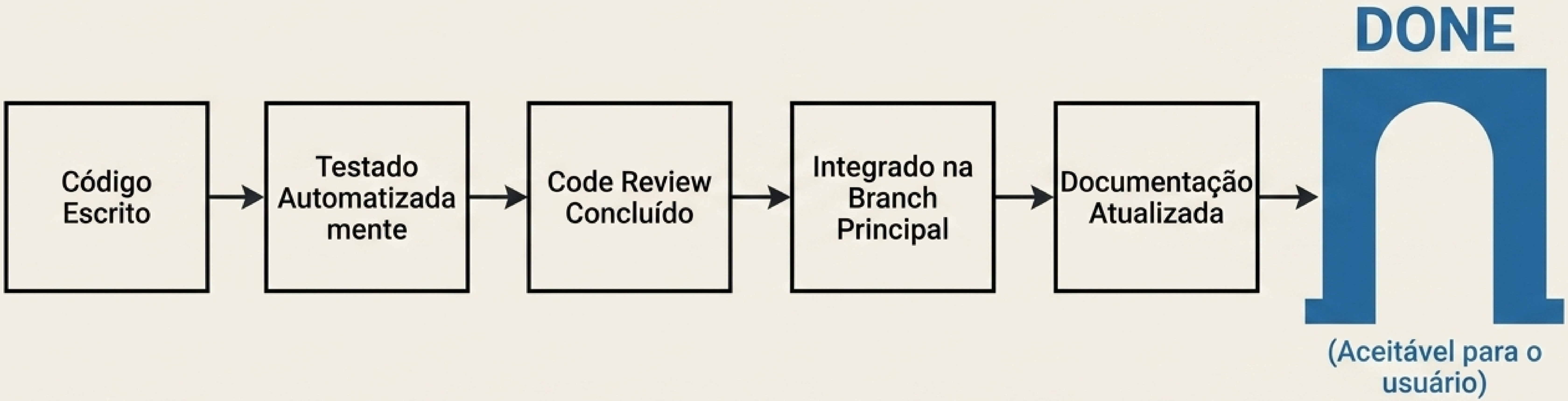
Execução e Qualidade. Decidem **como* transformar o trabalho.

Eficiência e Fluxo. Remove impedimentos do sistema.

Não é uma hierarquia. É uma unidade tática com responsabilidades complementares e um objetivo comum.

O Padrão de Conclusão: "Definition of Done"

A pergunta "Terminamos?" nunca deve ser respondida com "O código está escrito."



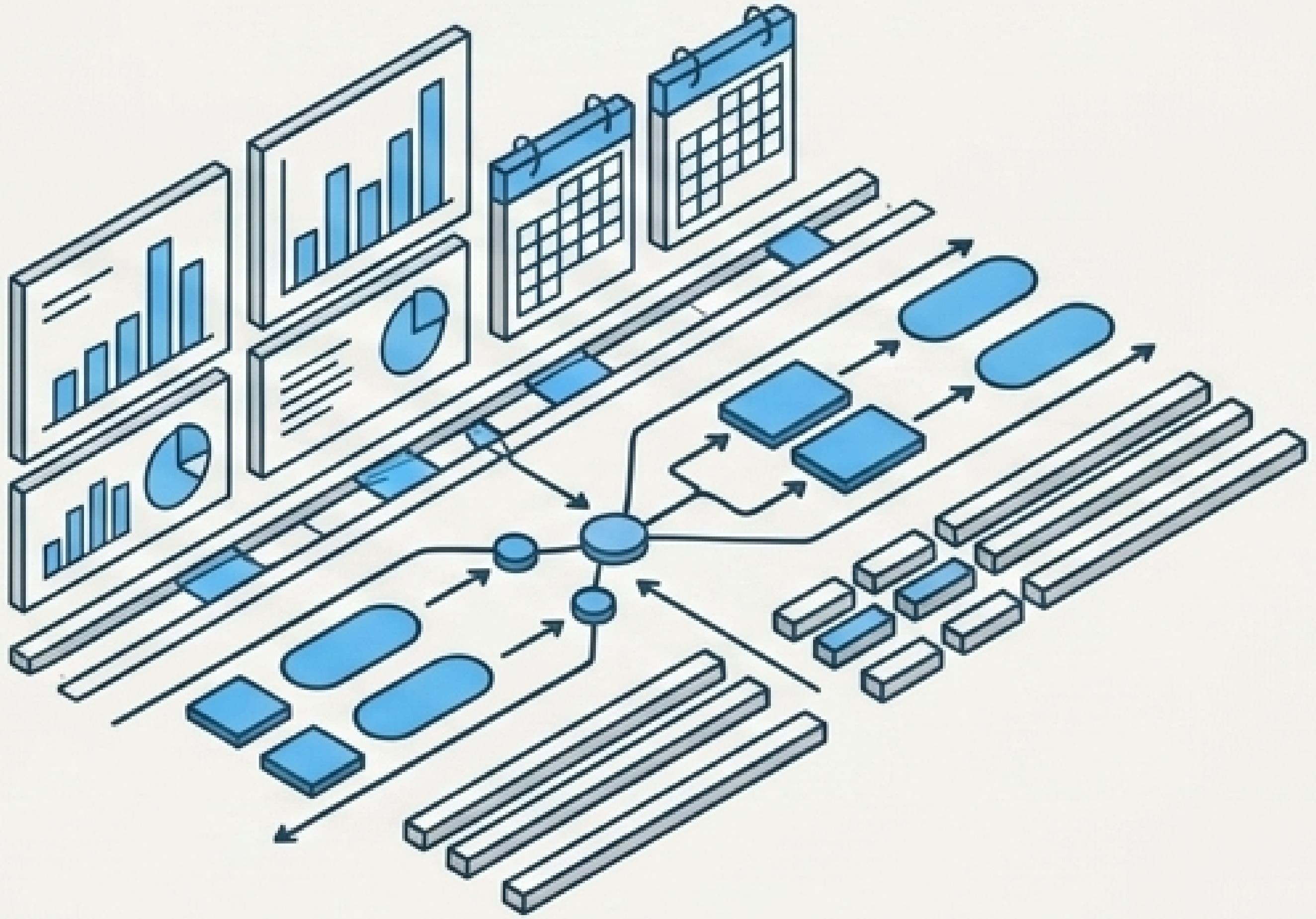
Código Escrito → Testado Automatizadamente → Code Review Concluído → Integrado na Branch Principal → Documentação Atualizada → DONE

(Aceitável para o usuário)

A **Definição de Pronto** garante que a equipe não está acumulando dívida técnica em nome da velocidade.

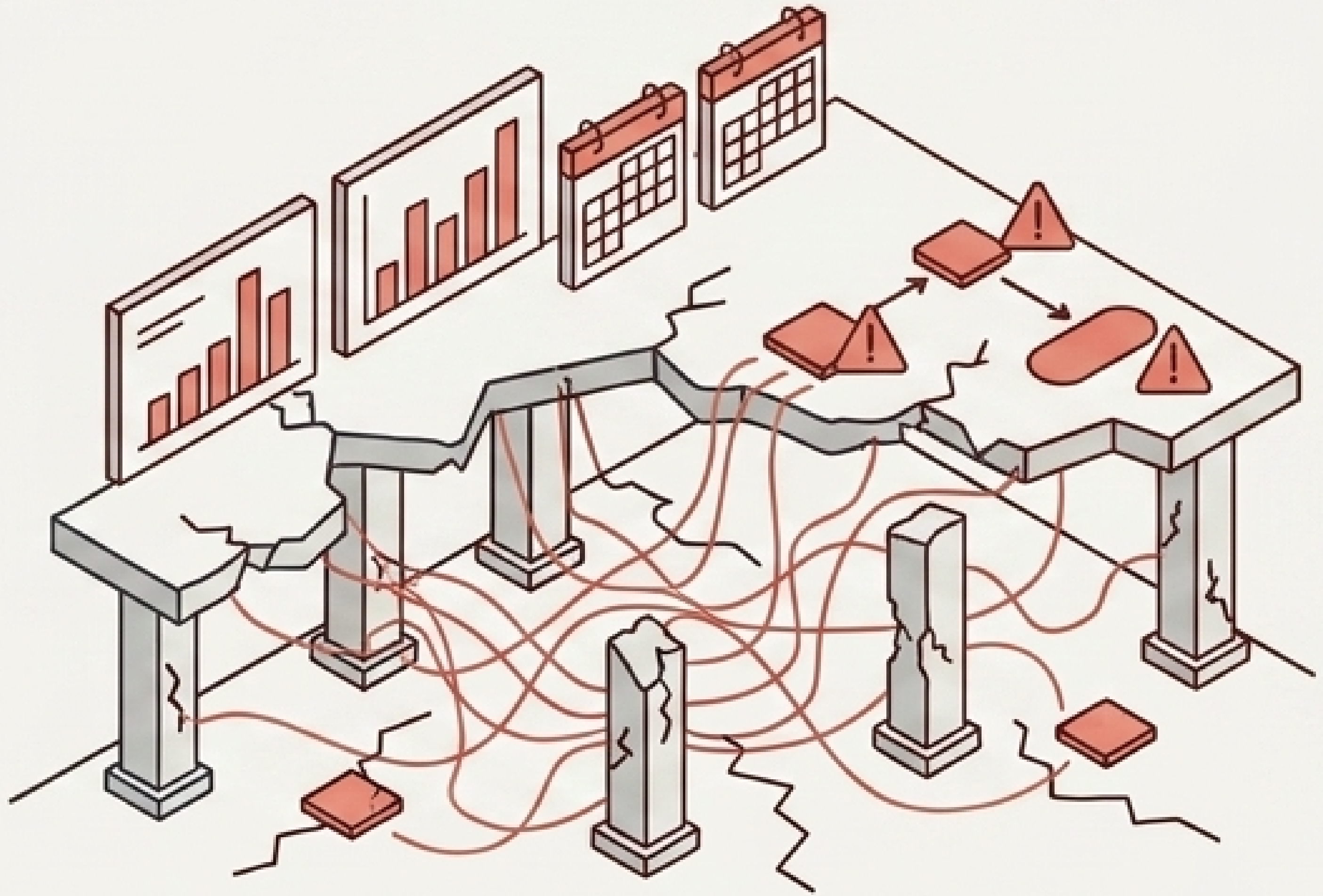
O Alerta de Engenharia

A Ilusão (O Processo Perfeito)



- Jiras organizados, Sprints fechadas e gráficos perfeitos.

A Realidade (A Base Técnica)

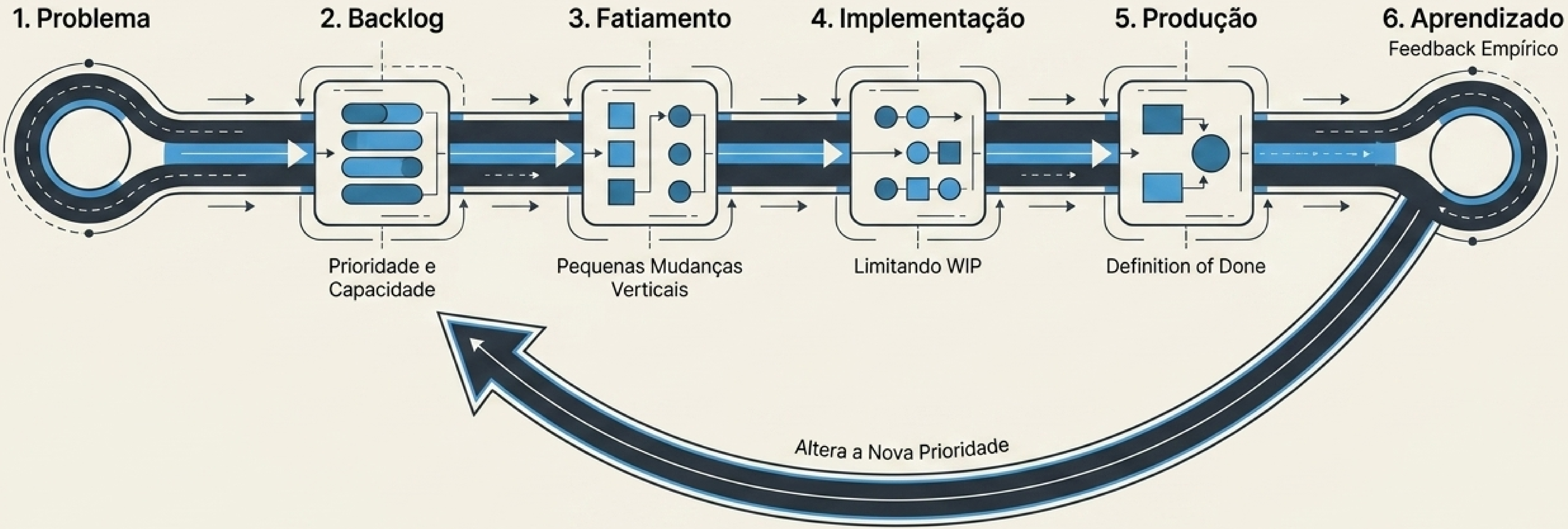


- Arquitetura ruim, PRs gigantes, deploys manuais e testes frágeis.

O Scrum organiza o trabalho. Ele não substitui boas práticas de engenharia. É possível fazer Scrum perfeitamente e ainda construir um software ruim.

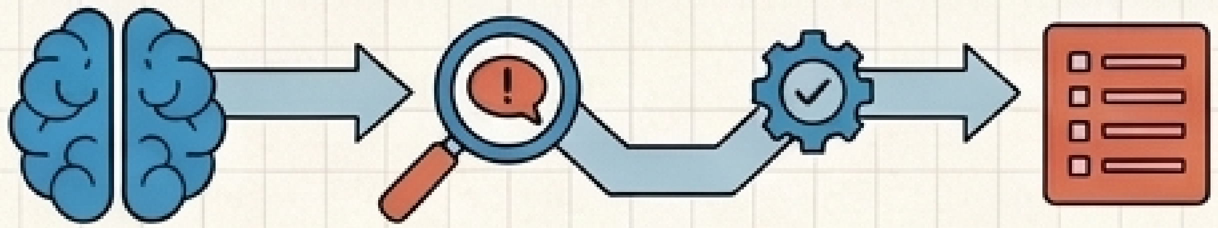
A Anatomia do Sistema Completo

O objetivo da engenharia não é eliminar esse ciclo, mas torná-lo **rápido, seguro e sustentável.**

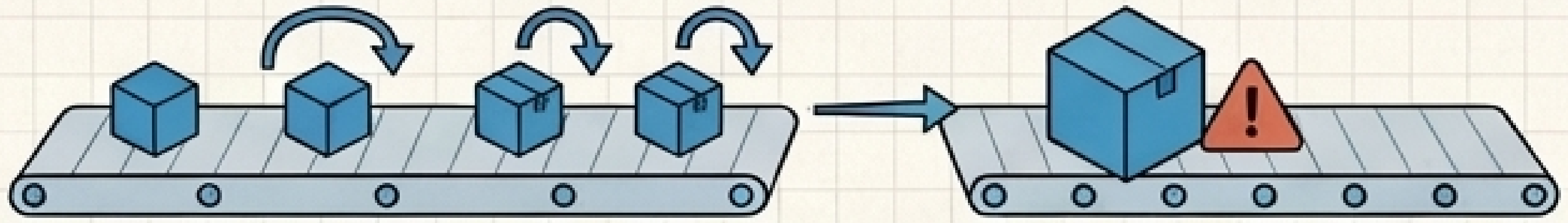

1. Problema
2. Backlog
3. Fatiamento
4. Implementação
5. Produção
6. Aprendizado
Feedback Empírico
Prioridade e Capacidade
Pequenas Mudanças Verticais
Limitando WIP
Definition of Done
Altera a Nova Prioridade

As 5 Diretrizes Operacionais

1. **Problemas > Tarefas.**
   O trabalho começa compreendendo o comportamento, não executando ordens.

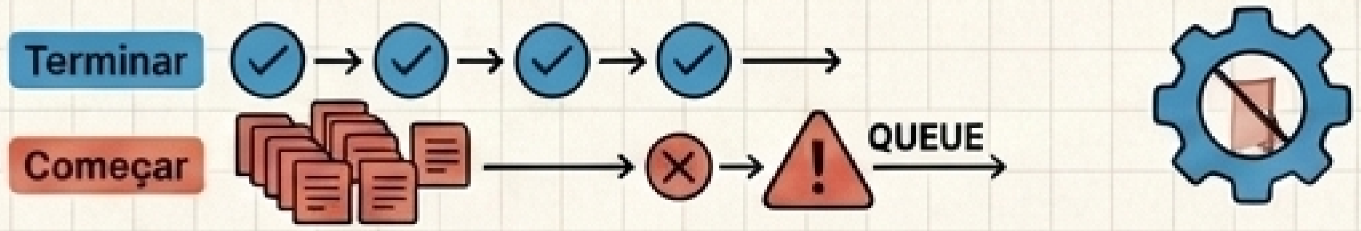


2. **Pequenos Lotes.**
   Mudanças pequenas reduzem risco e forçam feedback rápido.

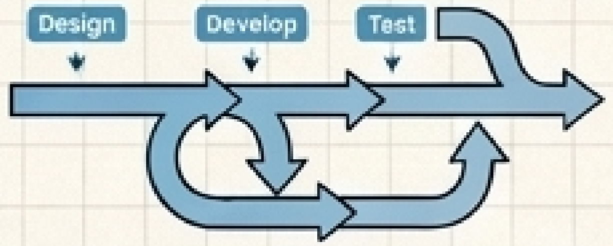


3. **Terminar > Começar.**
   WIP excessivo cria filas; feche ciclos antes de abrir novos.

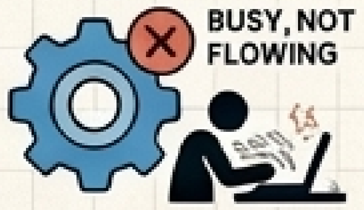
Terminar
Começar


Terminar
Começar
QUEUE

4. **Fluxo > Ocupação.**
   Otimize o sistema inteiro, não a digitação individual.


Design
Develop
Test

5. **Feedback Dita o Ritmo.**
   A equipe faz, observa, aprende e se adapta.


BUSY, NOT FLOWING

Uma equipe de software não é uma fábrica que processa requisitos.
É um sistema de aprendizagem contínua projetado para dominar a mudança.